

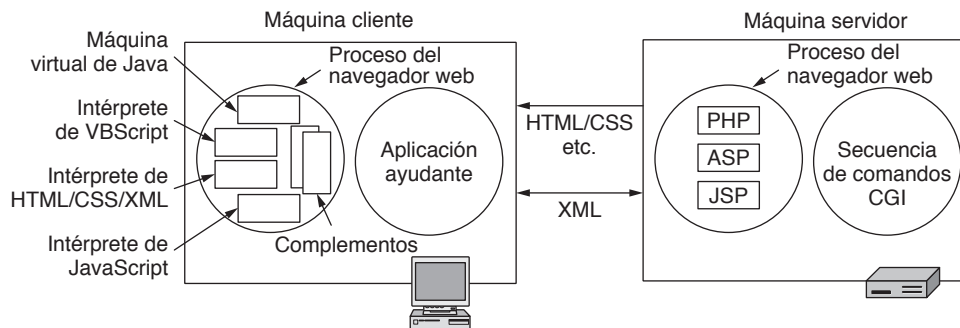


Figura 7-35. Varias tecnologías utilizadas para generar páginas dinámicas.

programas pueden realizar cálculos arbitrarios y actualizar la pantalla. Con AJAX, los programas en las páginas web pueden intercambiar XML y otros tipos de datos en forma asíncrona con el servidor. Este modelo soporta las aplicaciones web ricas en funcionalidad que se ven justo igual que las aplicaciones tradicionales, excepto que se ejecutan dentro del navegador y acceden a la información almacenada en servidores en Internet.

7.3.4 HTTP: el Protocolo de Transferencia de HiperTexto

Ahora que tenemos una idea sobre el contenido web y las aplicaciones, es tiempo de analizar el protocolo que se utiliza para transportar toda esta información entre servidores web y clientes: el **HTTP (Protocolo de Transferencia de HiperTexto)**, del inglés *HyperText Transfer Protocol*), según lo especificado en el RFC 2616.

HTTP es un protocolo simple de solicitud-respuesta que por lo general opera sobre TCP. Especifica qué mensajes pueden enviar los clientes a los servidores, y qué respuestas reciben de estos mensajes. Los encabezados de solicitud y respuesta se proporcionan en ASCII, justo igual que en SMTP. El contenido se proporciona en un formato parecido a MIME, también como el SMTP. Este modelo simple fue en parte responsable del éxito anticipado de la web, ya que simplificó los procesos de desarrollo e implementación.

En esta sección analizaremos las propiedades más importantes del HTTP y su uso en la actualidad. Sin embargo, antes de entrar en detalles cabe mencionar que la forma en que se utiliza en Internet está evolucionando. HTTP es un protocolo de la capa de aplicación, debido a que opera sobre TCP y está muy asociado con la web. Ésta es la razón por la que lo vemos en este capítulo. Sin embargo, en otro sentido el HTTP se está volviendo más parecido a un protocolo de transporte que provee los medios para que los procesos se comuniquen el contenido entre un límite y otro de las distintas redes. Estos procesos no tienen que ser un navegador y un servidor web. Un reproductor de medios podría usar HTTP para comunicarse con un servidor y solicitar información sobre un álbum. El software antivirus podría usarlo para descargar las actualizaciones más recientes. Los desarrolladores podrían usarlo para obtener los archivos de sus proyectos. Los productos de electrónica para el consumidor, como los marcos de fotografías digitales, utilizan con frecuencia un servidor HTTP incrustado como interfaz para el mundo exterior. La comunicación de máquina a máquina se realiza a través de HTTP cada vez con más frecuencia. Por ejemplo, un servidor de una aerolínea podría usar SOAP (una RPC de XML sobre HTTP) para contactarse con un servidor de renta de autos y reservar un vehículo, todo como parte de un paquete vacacional. Es probable que estas tendencias continúen, junto con el uso expandido de HTTP.

Conexiones

La forma común en que un navegador contacta a un servidor es mediante el establecimiento de una conexión TCP por el puerto 80 en la máquina del servidor, aunque este procedimiento no se requiere formalmente. El valor de utilizar TCP es que ni los navegadores ni los servidores tienen que preocuparse por cómo manejar los mensajes largos, la confiabilidad o el control de la congestión. Todos estos aspectos se manejan mediante la implementación de TCP.

En los primeros días de la web con el HTTP 1.0, una vez que se establecía la conexión, se enviaba una solicitud y se obtenía una respuesta. Después se liberaba la conexión TCP. Este método era adecuado en un mundo en el que una página web típica consistía por completo de texto HTML. Sin embargo, la página web promedio creció con rapidez y contenía una gran cantidad de vínculos incrustados para contenido tal como iconos y otros elementos visuales. En consecuencia, establecer una conexión TCP para transportar cada icono por separado era una manera muy costosa de operar.

Esta observación condujo al HTTP 1.1, que soporta **conexiones persistentes**. Con ellas es posible establecer una conexión TCP, enviar una solicitud y obtener una respuesta, para después enviar solicitudes adicionales y obtener respuestas adicionales. A esta estrategia se le conoce también como **reutilización de la conexión**. Al amortizar los costos de establecimiento, inicio y liberación de TCP entre varias solicitudes, se reduce la sobrecarga relativa debido a TCP por cada solicitud. También es posible canalizar solicitudes; es decir, enviar la solicitud 2 antes de que haya llegado la respuesta a la solicitud 1.

En la figura 7-36 se muestra la diferencia en el desempeño entre estos tres casos. La parte (a) muestra tres solicitudes, una después de la otra y cada una en una conexión separada. Supongamos que esto representa una página web con dos imágenes incrustadas en el mismo servidor. Los URL de las imágenes se determinan al momento de obtener la página principal, por lo que se obtienen después de ésta. En la actualidad, una página ordinaria tiene alrededor de 40 objetos distintos que se deben obtener para presentarla, pero eso aumentaría mucho nuestros números, por lo cual sólo usaremos dos objetos incrustados.

En la figura 7-36(b), la página se obtiene mediante una conexión persistente. Esto es, la conexión TCP se abre al inicio, después se envían las mismas tres solicitudes, una después de la otra como antes,

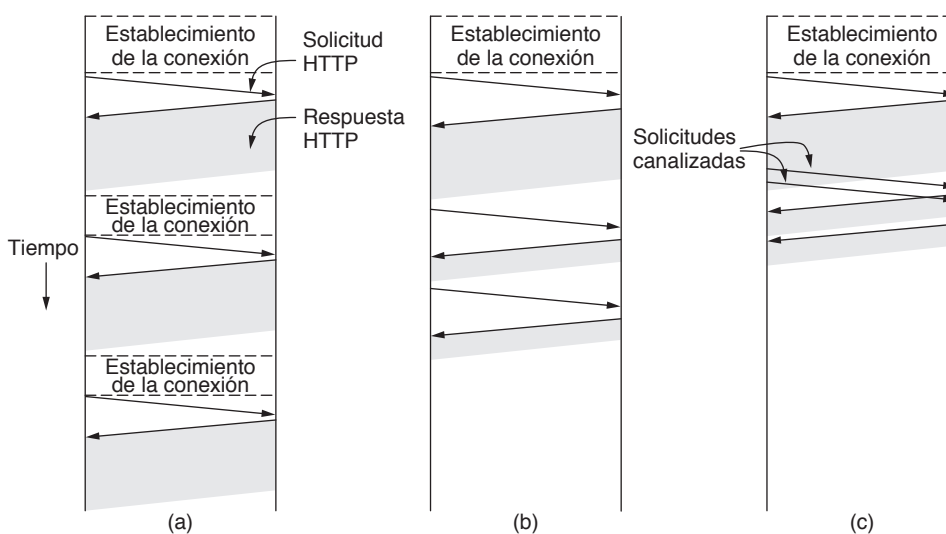


Figura 7-36. HTTP con (a) varias conexiones y solicitudes secuenciales. (b) Una conexión persistente y solicitudes secuenciales. (c) Una conexión persistente y solicitudes canalizadas.

y sólo entonces se cierra la conexión. Observe que la obtención se completa con más rapidez. Existen dos razones de la agilización. En primer lugar, no se desperdicia tiempo en establecer conexiones adicionales. Cada conexión TCP requiere cuando menos un tiempo de ida y vuelta para establecerse. En segundo lugar, la transferencia de la misma imagen se realiza con más rapidez. ¿Por qué es así? Se debe al control de la congestión de TCP. Al inicio de una conexión, TCP utiliza el procedimiento de inicio lento para incrementar la tasa de transferencia hasta que se aprende el comportamiento de la ruta de red. La consecuencia de este periodo de calentamiento es que varias conexiones TCP cortas tardan una mayor cantidad desproporcionada de tiempo en transferir información que una conexión TCP más larga.

Finalmente, en la figura 7-36(c) hay una conexión persistente y las solicitudes se canalizan. Específicamente, la segunda y la tercera se envían en rápida sucesión tan pronto como se haya recuperado lo suficiente de la página principal como para identificar que se deben obtener las imágenes. Con el transcurso del tiempo se envían las respuestas para estas solicitudes. Este método reduce el tiempo de inactividad del servidor, por lo que mejora el desempeño.

Sin embargo, las conexiones persistentes no son regaladas, puesto que surge un nuevo problema: cuándo cerrar la conexión. Una conexión a un servidor debe permanecer abierta mientras se carga la página. Entonces, ¿cuál es el problema? Hay una buena probabilidad de que el usuario haga clic en un vínculo que solicite otra página al servidor. Si la conexión permanece abierta, la siguiente solicitud se puede enviar de inmediato. Por lo tanto, no hay garantía de que el cliente vaya a hacer otra solicitud al servidor en cualquier momento. En la práctica, es común que los clientes y los servidores mantengan abiertas las conexiones persistentes hasta que hayan estado inactivos durante un intervalo corto (por ejemplo, 60 segundos), o cuando tengan una gran cantidad de conexiones abiertas y necesiten cerrar algunas.

El lector observador tal vez haya notado que hay una combinación de la que no hemos hablado hasta ahora. También es posible enviar una solicitud por cada conexión TCP, y ejecutar varias conexiones TCP en paralelo. Este método de **conexión paralela** se utilizaba mucho en los navegadores antes de las conexiones persistentes. Tiene la misma desventaja que las conexiones secuenciales: una sobrecarga adicional, pero a la vez ofrece un desempeño mucho mayor. Esto se debe a que al establecer y aumentar las conexiones en paralelo se oculta parte de la latencia. En nuestro ejemplo, las conexiones para las imágenes incrustadas se pueden establecer al mismo tiempo. Sin embargo, no se recomienda ejecutar muchas conexiones TCP con el mismo servidor. La razón es que TCP controla la congestión para cada conexión de manera independiente. Como consecuencia, las conexiones compiten entre sí, lo cual provoca una pérdida de paquetes adicional, y en conjunto son usuarios más agresivos de la red que una conexión individual. Las conexiones persistentes son superiores y es preferible usarlas en vez de las conexiones paralelas, ya que evitan una sobrecarga y no sufren de los problemas de congestión.

Métodos

Aunque HTTP se diseñó para utilizarlo en la web, se ha hecho intencionalmente más general de lo necesario con miras a futuros usos orientados a objetos. Por esta razón, se soportan otras operaciones, llamadas **métodos**, diferentes a las de solicitar una página web. Esta generalidad es lo que permite que SOAP exista.

Cada solicitud consiste en una o más líneas de texto ASCII; la primera palabra de la primera línea es el nombre del método solicitado. En la figura 7-37 se listan los métodos integrados. Los nombres son sensibles a mayúsculas y minúsculas, por lo que *GET* es un método válido y *get* no lo es.

El método *GET* solicita al servidor que envíe la página (cuando decimos “página” queremos decir “objeto”, en el caso más general, pero basta con pensar en una página como el contenido de un archivo

Método	Descripción
GET	Leer una página web.
HEAD	Leer el encabezado de una página web.
POST	Adjuntar a una página web.
PUT	Almacenar una página web.
DELETE	Eliminar la página web.
TRACE	Repetir la solicitud entrante
CONNECT	Conectarse a través de un proxy
OPTIONS	Consultar las opciones para una página

Figura 7-37. Los métodos de solicitud HTTP integrados.

para comprender los conceptos). La cual está codificada de forma adecuada en MIME. La gran mayoría de las solicitudes a servidores web son de tipo *GET*. La forma común de *GET* es:

GET nombreadarchivo HTTP/1.1

en donde *nombreadarchivo* nombra la página que se obtendrá y 1.1 es la versión del protocolo que se está utilizando.

El método *HEAD* sólo pide el encabezado del mensaje, sin la página real. Este método se puede utilizar para recolectar información para fines de indización, o sólo para evaluar la validez de un URL.

El método *POST* se utiliza para enviar formularios. Tanto este método como *GET* se emplean también para servicios web SOAP. Al igual que *GET*, porta también un URL, pero en lugar de sólo recuperar una página, envía datos al servidor (es decir, el contenido del formulario o los parámetros de RPC). Después el servidor hace algo con los datos que dependen del URL; conceptualmente adjunta los datos al objeto. El efecto podría ser comprar un elemento, por ejemplo, o llamar a un procedimiento. Finalmente, el método devuelve una página que indica el resultado.

El resto de los métodos no se utilizan mucho para navegar en la web. El método *PUT* es lo inverso a *GET*: en lugar de leer la página, la escribe. Este método hace posible la construcción de una colección de páginas web en un servidor remoto. El cuerpo de la solicitud contiene la página. Se puede codificar mediante el uso de MIME, en cuyo caso las líneas que siguen al método *PUT* podrían incluir encabezados de autenticación, para demostrar que el invocador realmente tiene permiso de realizar la operación solicitada.

DELETE hace lo que usted podría esperar: elimina la página, o al menos indica que el servidor web está de acuerdo con eliminar la página. Al igual que con *PUT*, la autenticación y el permiso juegan un papel importante aquí.

El método *TRACE* es para la depuración. Indica al servidor que regrese la solicitud. Este método es útil cuando las solicitudes no se están procesando de manera correcta y el cliente desea saber cuál solicitud ha obtenido realmente el servidor.

El método *CONNECT* permite a un usuario realizar una conexión con un servidor web por medio de un dispositivo intermedio, como una caché web.

El método *OPTIONS* proporciona una forma para que el cliente consulte al servidor sobre una página y obtenga los métodos y encabezados que se pueden usar con esa página.

Cada solicitud obtiene una respuesta que consiste en una línea de estado, y posiblemente de información adicional (por ejemplo, toda o parte de una página web). La línea de estado contiene un código de

estado de tres dígitos que indica si se atendió la solicitud, y si no, por qué. El primer dígito se utiliza para dividir las respuestas en cinco grupos principales, como se muestra en la figura 7-38. En la práctica, los códigos 1xx no se utilizan con frecuencia. Los códigos 2xx indican que la solicitud se manejó de manera exitosa y que se regresa el contenido (si hay alguno). Los códigos 3xx indican al cliente que busque en otro lado, ya sea mediante el uso de un URL diferente o en su propia caché (lo cual veremos más adelante). Los códigos 4xx significan que la solicitud falló debido a un error del cliente, por ejemplo: una solicitud inválida o una página inexistente. Por último, los errores 5xx indican que el servidor tiene un problema interno, debido a un error en su código o a una sobrecarga temporal.

Código	Significado	Ejemplos
1xx	Información	100 = el servidor acepta manejar la solicitud del cliente.
2xx	Éxito	200 = la solicitud es exitosa; 204 = no hay contenido.
3xx	Redirección	301 = se movió la página; 304 = la página en caché aún es válida.
4xx	Error del cliente	403 = página prohibida; 404 = no se encontró la página.
5xx	Error del servidor	500 = error interno del servidor; 503 = intentar más tarde.

Figura 7-38. Los grupos de respuesta del código de estado.

Encabezados de mensaje

A la línea de solicitud (por ejemplo, la línea con el método *GET*) le pueden seguir líneas adicionales que contienen más información. Estas se llaman **encabezados de solicitud**. Podemos comparar esta información con los parámetros de la llamada a un procedimiento. Las respuestas también pueden tener **encabezados de respuesta**. Algunos encabezados se pueden utilizar en cualquier dirección. En la figura 7-39 se muestra una selección de los más importantes. Esta lista no es corta, por lo que como podría imaginar, es común tener una variedad de encabezados en cada solicitud y respuesta.

El encabezado *User-Agent* permite que el cliente informe al servidor sobre la implementación de su navegador (por ejemplo, *Mozilla/5.0* y *Chrome/5.0.375.125*). Esta información es útil para que los servidores puedan ajustar sus respuestas para el navegador, ya que los distintos navegadores pueden tener herramientas y comportamientos que varíen de manera considerable.

Los cuatro encabezados *Accept* indican al servidor lo que el cliente está dispuesto a aceptar en caso de que tenga un repertorio limitado de lo que es aceptable. El primer encabezado especifica los tipos MIME aceptados (por ejemplo, *text/html*). El segundo proporciona el conjunto de caracteres (por ejemplo, *ISO-8859-5* o *Unicode-1-1*). El tercero tiene que ver con métodos de compresión (por ejemplo, *gzip*). El cuarto indica un idioma natural (por ejemplo, español). Si el servidor tiene varias páginas a elegir, puede utilizar esta información para proporcionar la que el cliente esté buscando. Si es incapaz de satisfacer la solicitud, se regresa un código de error y la solicitud falla.

Los encabezados *If-Modified-Since* e *If-None-Match* se utilizan con la caché. Permiten al cliente pedir que se envíe una página sólo si la copia en la caché ya no es válida. Más adelante hablaremos sobre el uso de la caché.

El encabezado *Host* da nombre al servidor. Este encabezado se toma del URL y es obligatorio. Se utiliza porque algunas direcciones IP pueden proporcionar varios nombres DNS y el servidor necesita una forma para saber a qué host debe entregar la solicitud.

El encabezado *Authorization* es necesario para páginas que están protegidas. En este caso, tal vez el cliente debe probar que tiene permiso para ver la página solicitada. Este encabezado se utiliza para ese caso.

Encabezado	Tipo	Contenidos
User-Agent	Solicitud	Información sobre el navegador y su plataforma.
Accept	Solicitud	El tipo de páginas que puede manejar el cliente.
Accept-Charset	Solicitud	Los conjuntos de caracteres que son aceptables para el cliente.
Accept-Encoding	Solicitud	Las codificaciones de página que puede manejar el cliente.
Accept-Language	Solicitud	Los idiomas naturales que puede manejar el cliente.
If-Modified-Since	Solicitud	Hora y fecha para verificar la actualidad de un mensaje.
If-None-Match	Solicitud	Etiquetas enviadas previamente para verificar la actualidad de un mensaje.
Host	Solicitud	El nombre DNS del servidor.
Authorization	Solicitud	Una lista de las credenciales del cliente.
Referer	Solicitud	El URL anterior desde el cual provino la solicitud.
Cookie	Solicitud	La cookie establecida previamente que se regresa al servidor.
Set-Cookie	Respuesta	La cookie que debe guardar el cliente.
Server	Respuesta	Información sobre el servidor.
Content-Encoding	Respuesta	Cómo se codifica el contenido (por ejemplo, gzip).
Content-Language	Respuesta	El lenguaje natural utilizado en la página.
Content-Length	Respuesta	La longitud de la página en bytes.
Content-Type	Respuesta	El tipo MIME de la página.
Content-Range	Respuesta	Identifica una parte del contenido de la página.
Last-Modified	Respuesta	Hora y fecha de la última modificación de la página.
Expires	Respuesta	Hora y fecha en que la página dejará de ser válida.
Location	Respuesta	Indica al cliente a dónde enviar su solicitud.
Accept-Ranges	Respuesta	Indica que el servidor aceptará solicitudes de rango de bytes.
Date	Ambas	Fecha y hora en que se envió el mensaje.
Range	Ambas	Identifica una parte de una página.
Cache-Control	Ambas	Directivas para manejar las cachés.
ETag	Ambas	Etiqueta para el contenido de la página.
Upgrade	Ambas	El protocolo al que el emisor desea conmutar.

Figura 7-39. Algunos encabezados de mensaje HTTP.

El cliente usa el encabezado *Referer* mal escrito para proporcionar el URL que hizo referencia al URL que ahora se solicita. Lo más común es que sea el URL de la página anterior. Este encabezado es particularmente útil para rastrear la navegación web, ya que indica a los servidores cómo llegó un cliente a la página.

Aunque las cookies se tratan en el RFC 2109 en lugar de en el RFC 2616, también tienen encabezados. El encabezado *Set-Cookie* es la forma en que los servidores envían cookies a los clientes. Éste debe

guardar la cookie y regresarla en las solicitudes posteriores al servidor, mediante el uso del encabezado *Cookie* (cabe mencionar que hay una especificación más reciente para las cookies con encabezados más recientes, conocida como RFC 2965, pero la industria la rechazó en gran parte, por lo cual no se implementa mucho).

En las respuestas se usan muchos otros encabezados. El encabezado *Server* permite que el servidor identifique su versión de compilación de software si lo desea. Los siguientes cinco encabezados, que empiezan todos con *Content-*, permiten al servidor describir las propiedades de la página que va a enviar.

El encabezado *Last-Modified* indica cuándo se modificó la página por última vez y *Expires* indica por cuánto tiempo será válida la página. Ambos encabezados desempeñan un papel importante en el uso de la caché para las páginas.

El servidor utiliza el encabezado *Location* para informar al cliente que debe probar con un URL distinto. Esto se puede usar si se cambió la ubicación de la página o para permitir que varios URL hagan referencia a la misma página (posiblemente en distintos servidores). También se utiliza para las empresas que tienen una página web principal en el dominio *com*, pero redirigen a sus clientes a una página nacional o regional con base en sus direcciones IP o su idioma preferido.

Si una página es muy grande, tal vez un cliente pequeño no quiera recibirla toda de una sola vez. Algunos servidores aceptan solicitudes de rangos de bytes, de modo que la página se pueda obtener en varias unidades pequeñas. El encabezado *Accept-Ranges* anuncia la disposición del servidor para manejar este tipo de solicitud parcial de página.

Ahora veamos los encabezados que se pueden usar en ambas direcciones. *Date* se puede usar en ambas direcciones y contiene la fecha y hora en que se envió el mensaje, mientras que *Range* indica el rango de bytes de la página que se provee mediante la respuesta.

El encabezado *ETag* proporciona una etiqueta corta que sirve como nombre para el contenido de la página. Se utiliza para trabajar con la caché. El encabezado *Cache-Control* proporciona otras instrucciones explícitas sobre cómo colocar páginas en la caché (o, lo que es más común, cómo no colocarlas).

Por último, el encabezado *Upgrade* se utiliza para conmutar a un nuevo protocolo de comunicación, como un protocolo HTTP futuro o un transporte seguro. Este encabezado permite al cliente anunciar lo que puede soportar, y al servidor afirmar lo que está usando.

Almacenamiento en caché

A menudo las personas regresan a las páginas web que han visto antes, y es común que las páginas web relacionadas tengan los mismos recursos incrustados. Algunos ejemplos son las imágenes que se utilizan para navegar a través del sitio, así como las hojas de estilo y las secuencias de comandos comunes. Sería un desperdicio obtener todos estos recursos para esas páginas cada vez que se desplieguen en pantalla, puesto que el navegador ya tiene una copia.

Al proceso de guardar y ocultar las páginas que se obtienen para usarlas después se le conoce como **almacenamiento en caché**. La ventaja es que, cuando se puede reutilizar una página en caché, no es necesario repetir la transferencia. HTTP tiene soporte integrado para ayudar a los clientes a identificar cuándo pueden reutilizar páginas. Este soporte mejora el desempeño al reducir tanto el tráfico como la latencia en la red. La desventaja es que el navegador ahora debe guardar páginas, pero esto casi siempre vale la pena, ya que el almacenamiento local no es muy costoso. Por lo general las páginas se mantienen en el disco, de modo que se puedan usar al ejecutar el navegador en una fecha posterior.

El verdadero problema con el almacenamiento en caché en HTTP es cómo determinar que una copia de una página previamente colocada en caché es la misma que se obtendría del servidor otra vez. No podemos hacer esta determinación únicamente con base en el URL. Por ejemplo, tal vez el URL

puede proporcionar una página que muestra la noticia más reciente. El contenido de esta página se actualizará con frecuencia, incluso cuando el URL permanezca igual. O por el contrario, el contenido de la página puede ser una lista de los dioses de la mitología griega y romana. Lo más seguro es que esta página cambie con menos rapidez.

HTTP utiliza dos estrategias para lidiar con este problema, las cuales se muestran en la figura 7-40 como formas de procesamiento entre la solicitud (paso 1) y la respuesta (paso 5). La primera estrategia es la validación de páginas (paso 2). Se consulta la caché y, si tiene una copia de una página para el URL solicitado que se sabe está actualizada (es decir, aún es válida), no hay necesidad de obtenerla de nuevo del servidor. En cambio, se puede regresar la página en caché directamente. Para realizar esta determinación podemos utilizar el encabezado *Expires* que se devolvió cuando se obtuvo por primera vez la página en caché, junto con la fecha y hora actuales.

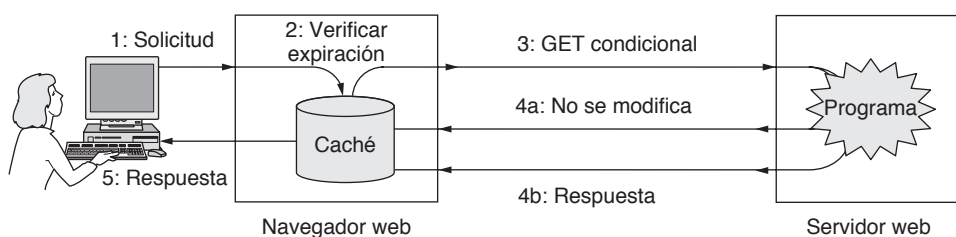


Figura 7-40. Almacenamiento en caché en HTTP.

Sin embargo, no todas las páginas incluyen un encabezado *Expires* apropiado que les indica cuándo debe obtener la página nuevamente. Después de todo, es difícil hacer predicciones —en especial sobre el futuro. En este caso, el navegador puede usar la heurística. Por ejemplo, si la página no se ha modificado en el último año (según lo que indica el encabezado *Last-Modified*), es de suponer que no cambiará en la siguiente hora. Por lo tanto, no hay garantía, por lo que ésta podría ser una mala suposición. Por ejemplo, el mercado de valores podría haber cerrado sus operaciones del día, de modo que la página no cambiará durante horas, pero cambiará con rapidez una vez que inicie la siguiente sesión de compra-venta de valores. Por ende, la factibilidad de almacenar una página en caché variará de manera considerable con el tiempo. Por esta razón se debe usar la heurística con cuidado, aunque por lo general funciona bien en la práctica.

Encontrar páginas que no hayan expirado es el uso más benéfico del almacenamiento en caché, ya que significa que no hay que tener ningún contacto con el servidor. Por desgracia, esto no siempre funciona. Los servidores deben usar el encabezado *Expires* de manera conservadora, ya que tal vez no estén seguros acerca de cuándo se actualizará una página. Por lo tanto, las copias en caché tal vez estén actualizadas pero el cliente no lo sepa.

En este caso se utiliza la segunda estrategia: preguntar al servidor si la copia en caché sigue siendo válida. Esta solicitud es un **GET condicional** y se muestra en la figura 7-40 como el paso 3. Si el servidor sabe que la copia en caché aún es válida, puede enviar una respuesta corta para decirlo (paso 4a). En caso contrario, debe enviar la respuesta completa (paso 4b).

Se utilizan más campos de encabezado para que el servidor pueda verificar si una copia en caché aún es válida. El cliente tiene la hora en que se actualizó una página en caché por última vez, a través del encabezado *Last-Modified*. Puede enviar esta hora al servidor mediante el uso del encabezado *If-Modified-Since* para pedir la página sólo si se modificó en ese tiempo.

Como alternativa, el servidor puede devolver un encabezado *ETag* con una página. Este encabezado proporciona una etiqueta que es un nombre corto para el contenido de la página, como una suma de

verificación, sólo que mejor (puede ser un *hash* criptográfico, el cual describiremos en el capítulo 8). Para validar las copias en caché, el cliente envía un encabezado *If-None-Match* con una lista de las etiquetas. Si alguna de ellas coincide con el contenido con el que el servidor respondería, se puede usar la correspondiente copia en caché. Podemos utilizar este método cuando no sea conveniente ni útil determinar la actualidad de una página. Por ejemplo, un servidor puede regresar distinto contenido para el mismo URL, dependiendo de los idiomas y tipos MIME que se prefieran. En este caso, la fecha de modificación por sí sola no ayudará al servidor a determinar si la página en caché es actual.

Por último, observe que ambas estrategias de almacenamiento en caché son anuladas por las directivas que se transportan en el encabezado *Cache-Control*. Estas directivas se pueden usar para restringir el almacenamiento en caché (por ejemplo, *no-cache*) cuando no sea apropiado. Un ejemplo es una página dinámica, que será diferente la próxima vez que se obtenga. Las páginas que requieren autorización tampoco se almacenan en caché.

Hay mucho más en cuanto al almacenamiento en caché, pero sólo tenemos el espacio suficiente para expresar dos puntos importantes. En primer lugar, el almacenamiento en caché se puede realizar en otros lugares además del navegador. En el caso general, las solicitudes HTTP se pueden enrutar a través de una serie de cachés. El uso de una caché externa al navegador se conoce como **almacenamiento en caché proxy**. Cada nivel agregado de almacenamiento en caché puede ayudar a reducir las solicitudes en niveles superiores de la cadena. Es común que las organizaciones como los ISP y las empresas ejecuten cachés proxy para obtener los beneficios de almacenar páginas en caché entre distintos usuarios. En la sección 7.5, al final de este capítulo, analizaremos el almacenamiento en caché proxy con el tema más amplio de la distribución de contenido.

En segundo lugar, las cachés proveen un aumento importante en el desempeño, pero no tanto como podríamos esperar. La razón es que, aunque hay ciertos documentos populares en la web, también hay muchos documentos no tan populares que las personas obtienen, muchos de los cuales son también muy extensos (por ejemplo, los videos). La “cola larga” de los documentos no populares ocupa espacio en las cachés, y la cantidad de solicitudes que se pueden manejar desde la caché aumenta de manera lenta con el tamaño de la caché. Siempre existe la probabilidad de que las cachés web puedan manejar menos de la mitad de las solicitudes. Consulte a Breslau y colaboradores (1999) para obtener más información.

Experimentación con el HTTP

Puesto que el HTTP es un protocolo ASCII, es bastante fácil para una persona en una terminal (lo opuesto a un navegador) comunicarse de manera directa con los servidores web. Todo lo que se necesita es una conexión TCP al puerto 80 en el servidor. Se anima a los lectores a experimentar con la siguiente secuencia de comandos. Funcionará en la mayoría de los *shells* de UNIX y en la ventana de comandos de Windows (una vez que se habilite el programa telnet).

```
telnet www.ietf.org 80
GET /rfc.html HTTP/1.1
Host: www.ietf.org
```

Esta secuencia de comandos inicia una conexión telnet (es decir, TCP) al puerto 80 en el servidor web de la IETF, *www.ietf.org*. Después viene el comando *GET* que nombra la ruta del URL y el protocolo. Pruebe con servidores y direcciones URL de su elección. La siguiente línea es el encabezado *Host* obligatorio. Una línea en blanco después del último encabezado es obligatoria, ya que indica al servidor que no hay más encabezados de solicitud. A continuación el servidor enviará la respuesta. Dependiendo del servidor y el URL, se pueden observar muchos tipos distintos de encabezados y páginas.

7.3.5 La web móvil

La Web se utiliza desde casi cualquier tipo de computadora, incluyendo los teléfonos móviles. Puede ser muy útil navegar por la web a través de una red inalámbrica mientras nos desplazamos de un lugar a otro. También se presentan problemas técnicos, ya que gran parte del contenido web está diseñado para presentaciones ostentosas en computadoras de escritorio con conectividad de banda ancha. En esta sección describiremos cómo se está desarrollando el acceso web desde dispositivos móviles, mejor conocido como **web móvil**.

En comparación con las computadoras de escritorio en el trabajo o el hogar, los teléfonos móviles presentan varias dificultades para la navegación web:

1. Las pantallas relativamente pequeñas impiden ver páginas extensas e imágenes grandes.
2. Las capacidades limitadas de entrada hacen que sea tedioso introducir direcciones URL u otro tipo de entrada extensa.
3. El ancho de banda de red está limitado a través de los enlaces inalámbricos, en especial en las redes celulares (3G), en donde con frecuencia también es costoso.
4. La conectividad puede ser intermitente.
5. El poder de cómputo está limitado por cuestiones de vida de la batería, tamaño, disipación de calor y costo.

Estas dificultades significan que hay una gran probabilidad de que el simple uso de contenido de escritorio para la web móvil provoque una experiencia frustrante en el usuario.

En las primeras metodologías para la web móvil se ideó una nueva pila de protocolos enfocada a los dispositivos inalámbricos con capacidades limitadas. **WAP (Protocolo de Aplicaciones Inalámbricas**, del inglés *Wireless Application Protocol*) es el ejemplo más conocido de esta estrategia. El esfuerzo de WAP lo empezaron en 1997 los principales distribuidores de teléfonos móviles: Nokia, Ericsson y Motorola. Sin embargo, ocurrió algo inesperado en el camino. En la siguiente década, el ancho de banda de red y las capacidades de los dispositivos aumentaron de manera considerable gracias a la implementación de los servicios de datos 3G y los teléfonos móviles con pantallas más grandes a color, procesadores más rápidos y capacidades inalámbricas 802.11. De repente los teléfonos móviles ya eran capaces de ejecutar navegadores web simples. Aún hay un vacío entre estos teléfonos móviles y los equipos de escritorio que nunca se cerrará, pero han desaparecido muchos de los problemas tecnológicos que dieron ímpetu al tema de una pila de protocolos separada.

El método que se utiliza cada vez con más frecuencia es ejecutar los mismos protocolos web para los equipos móviles y los de escritorio, y hacer que los sitios web ofrezcan contenido amigable para los móviles cuando el usuario se encuentre en un dispositivo de este tipo. Los servidores web son capaces de detectar si deben regresar versiones de escritorio o móviles de las páginas web con sólo analizar los encabezados de la solicitud. El encabezado *User-Agent* es en especial útil en esta cuestión, ya que identifica el software del navegador. Así, cuando un servidor web recibe una solicitud, puede analizar los encabezados y regresar una página con imágenes pequeñas, menos texto y una navegación más simple para un iPhone, y una página completa al usuario en una computadora portátil.

El W3C está fomentando esta metodología de varias formas. Una de ellas es estandarizar las mejores prácticas para el contenido web móvil. En la primera especificación se provee una lista de 60 de esas mejores prácticas (Rabin y McCathieNevile, 2008). La mayoría de estas prácticas llevan a cabo pasos prudentes para reducir el tamaño de las páginas, incluyendo el uso de la compresión, ya que los costos de comunicación son más altos que los del poder de cómputo; también se maximiza la efectividad del almacenamiento en caché. Esta metodología anima a los sitios, en especial los grandes, a que creen versiones web móviles de su contenido, ya que es todo lo que necesitan para capturar a los usuarios de la web móvil.

Para ayudar a esos usuarios en el proceso, también existe un logo que indica las páginas que se pueden ver (bien) en la web móvil.

Otra herramienta útil es una versión simplificada de HTML, conocida como **XHTML Básico**. Este lenguaje es un subconjunto de XHTML destinado para usarse en teléfonos móviles, televisiones, dispositivos PDA, máquinas expendedoras, radiolocalizadores, autos, máquinas de juegos e incluso hasta relojes. Por esta razón no soporta las hojas de estilo, las secuencias de comandos ni los marcos, pero la mayoría de las etiquetas estándar están ahí. Se agrupan en 11 módulos. Algunas son requeridas; otras son opcionales. Todas están definidas en XML. En la figura 7-41 se listan los módulos y algunas etiquetas de ejemplo.

Módulo	¿Requerido?	Función	Etiquetas de ejemplo
Estructura	Sí	Estructura del documento.	body, head, html, title
Texto	Sí	Información.	br, code, dfn, em, hn, kbd, p, strong
Hipertexto	Sí	Hipervínculos.	a
Lista	Sí	Listas detalladas.	dl, dt, dd, ol, ul, li
Formularios	No	Formularios de entrada de datos.	form, input, label, option, textarea
Tablas	No	Tablas rectangulares.	caption, table, td, th, tr
Imagen	No	Imágenes.	img
Objeto	No	Applets, mapas, etcétera.	object, param
Metainformación	No	Información extra.	meta
Vínculo	No	Similar a <a>.	link
Base	No	Punto de inicio del URL.	base

Figura 7-41. Los módulos y etiquetas de XHTML Básico.

Sin embargo, no todas las páginas estarán diseñadas para trabajar bien en la web móvil. Por lo tanto, una metodología complementaria es el uso de la **transformación de contenido o transcodificación**. En esta metodología, una computadora que está entre el móvil y el servidor recibe las solicitudes del móvil, obtiene el contenido del servidor y lo transforma en contenido web móvil. Una transformación simple es reducir el tamaño de las imágenes grandes, para lo cual se ajusta el formato para producir una resolución más baja. Se pueden usar muchas otras transformaciones pequeñas pero útiles. La transcodificación se ha utilizado con cierto éxito desde los primeros días de la web móvil. Consulte, por ejemplo, a Fox y colaboradores (1996). Sin embargo, cuando se utilizan ambas metodologías hay una tensión entre las decisiones de contenido móvil que toma el servidor y las que toma el transcodificador. Por ejemplo, tal vez un sitio web seleccione una combinación específica de imagen y texto para un usuario de la web móvil, sólo para que un transcodificador cambie el formato de la imagen.

Hasta ahora nuestra discusión ha sido sobre el contenido, no los protocolos, ya que éste es el mayor problema para realizar la web móvil. Sin embargo, mencionaremos con brevedad la cuestión de los protocolos. En la web, el uso de los protocolos HTTP, TCP e IP puede consumir una cantidad considerable de ancho de banda debido a las sobrecargas de los protocolos, como los encabezados. Para lidiar con este problema, WAP y otras soluciones definieron protocolos de propósito especial. Esto resulta ser en gran parte innecesario. Las tecnologías de compresión de encabezados, como la **ROHC**

(**Compresión RObusta de Encabezados**, del inglés *RObust Header Compression*) que se describe en el capítulo 6, pueden reducir las sobrecargas de estos protocolos. De esta forma, es posible tener un conjunto de protocolos (HTTP, TCP, IP) y utilizarlos a través de enlaces de ancho de banda alto y bajo. El uso a través de los enlaces de bajo ancho de banda simplemente requiere la activación de la compresión de los encabezados.

7.3.6 Búsqueda web

Para terminar nuestra descripción de la web, analizaremos lo que sin duda es la aplicación web más exitosa: la búsqueda. En 1998, Sergey Brin y Larry Page, que entonces eran estudiantes de maestría en Stanford, formaron una empresa nueva llamada Google para construir un mejor motor de búsqueda web. Estaban armados con la entonces radical idea de que un algoritmo de búsqueda que contara cuántas veces una página era apuntada por otras representaba una mejor medida de su importancia, en vez de contar cuántas veces contenía las palabras clave que se estaban buscando. Por ejemplo, muchas páginas tienen vínculos hacia la página principal de Cisco, lo cual hace a ésta más importante para un usuario que busca “Cisco” que una página que no esté relacionada con la empresa y simplemente utilice la palabra “Cisco” muchas veces.

Ellos tenían razón. Se demostró que era posible construir un mejor motor de búsqueda y las personas se apresuraron a usarlo. Gracias al respaldo del capital de riesgo, Google creció de manera considerable. Se convirtió en empresa pública en 2004, con una capitalización de mercado de \$23 mil millones. Para 2010 se estimó que ejecutaba más de un millón de servidores en centros de datos esparcidos por todo el mundo.

En cierto sentido, la búsqueda es simplemente otra aplicación web, aunque es una de las más maduras debido a que ha estado en desarrollo desde los primeros días de la web. Sin embargo, la búsqueda web ha demostrado ser indispensable en el uso diario. Se estima que se realizan más de mil millones de búsquedas en la web cada día. Las personas que buscan todo tipo de información usan la búsqueda como punto inicial. Por ejemplo, para averiguar en dónde comprar Vegemite en Seattle, no hay un sitio web obvio que podamos usar como punto de partida. Pero es probable que un motor de búsqueda conozca una página con la información deseada y nos pueda llevar con rapidez a la respuesta.

Para realizar una búsqueda web de la manera tradicional, el usuario dirige su navegador al URL de un sitio web de búsqueda. Los principales sitios de búsqueda son: Google, Yahoo! y Bing. A continuación el usuario envía los términos de búsqueda mediante el uso de un formulario. Esta acción provoca que el motor de búsqueda realice una consulta en su base de datos en cuanto a páginas o imágenes relevantes, o cualquier tipo de recurso que se esté buscando, y que devuelva el resultado como una página dinámica. Así, el usuario puede seguir vínculos a las páginas que se hayan encontrado.

La búsqueda web es un interesante tema de discusión, ya que existen implicaciones en cuanto al diseño y el uso de las redes. En primer lugar, está la pregunta de cómo la búsqueda web encuentra las páginas. El motor de búsqueda web debe tener una base de datos de páginas para ejecutar una consulta. Cada página de HTML puede contener vínculos a otras páginas, y todo lo interesante (o por lo menos que se pueda buscar) está vinculado en alguna otra parte. Esto significa que en teoría es posible iniciar con un puñado de páginas y encontrar a todas las demás páginas en la web mediante un recorrido de todas las páginas y vínculos. A este proceso se le conoce como **Rastreo Web** (*Web Crawling*). Todos los motores de búsqueda web usan rastreador web.

Un problema con el rastreo web es el tipo de páginas que puede encontrar. Es fácil obtener documentos estáticos y seguir los vínculos. Sin embargo, muchas páginas web contienen programas que muestran distintas páginas, dependiendo de la interacción del usuario. Como ejemplo está el catálogo en línea de una tienda. Este catálogo puede contener páginas dinámicas creadas a partir de una base de datos y consultas sobre distintos productos. Este tipo de contenido es diferente a las páginas estáticas que son

fáciles de recorrer. ¿Cómo encuentran los rastreadores web estas páginas dinámicas? La respuesta es que, la mayoría, no lo hacen. A este tipo de contenido oculto se le conoce como **web profunda**. La forma de buscar en la web profunda es un problema abierto que los investigadores están tratando de resolver. Por ejemplo, consulte a Madhavan y colaboradores (2008). Existen también convenciones mediante las cuales los sitios crean una página (conocida como *robots.txt*) para indicar a los rastreadores qué partes de los sitios deben o no visitar.

Una segunda consideración es cómo procesar todos los datos examinados. Para permitir que se ejecuten algoritmos de indexado sobre la masa de datos, hay que almacenar las páginas. Las estimaciones varían, pero se piensa que los principales motores de búsqueda tienen un índice de decenas de miles de millones de páginas que se toman de la parte visible de la web. El tamaño de página promedio se estima en 320 KB. Estas cifras estiman que una copia rastreada de la web tarda cerca de 20 petabytes o 2×10^{16} bytes en almacenarse. Aunque éste es un número enorme, es también una cantidad de datos que se puede almacenar y procesar sin problemas en los centros de datos de Internet (Chang y colaboradores, 2006). Por ejemplo, si el almacenamiento en disco cuesta \$20/TB, entonces 2×10^4 TB cuestan \$400 000, lo cual no es en sí una cantidad grande para empresas del tamaño de Google, Microsoft y Yahoo! Y mientras se expande la web, los costos de los discos disminuyen en forma drástica, por lo que almacenar toda la web completa puede seguir siendo algo viable para las empresas grandes en el futuro inmediato.

Hacer uso de estos datos es otro asunto. Aquí se puede apreciar cómo puede el XML ayudar a los programas a extraer la estructura de los datos con facilidad, mientras que los formatos específicos conducirán a muchas conjeturas. Existe también el problema de la conversión entre los formatos, e incluso de la traducción entre los lenguajes. Pero incluso conocer la estructura de los datos es sólo parte del problema. Lo difícil es entender su significado. Aquí es donde se puede descubrir mucho valor, empezando con las páginas de resultados más relevantes para las consultas de búsqueda. El objetivo más importante es responder a las preguntas; por ejemplo, dónde comprar un tostador económico pero decente en su ciudad.

Un tercer aspecto de la búsqueda web es que ha llegado a proveer un nivel más alto en cuanto a los nombres. No hay necesidad de recordar un URL largo si es igual de confiable (o tal vez más) buscar una página web con base en el nombre de la persona, suponiendo que seamos mejor al recordar nombres en vez de los URL. Esta estrategia está teniendo cada vez más éxito. De la misma forma que los nombres DNS relegan direcciones IP a las computadoras, la búsqueda web relega direcciones URL a las computadoras. Otra cosa a favor de la búsqueda es que corrige los errores de ortografía y de escritura, mientras que si escribimos un URL mal, recibimos la página incorrecta.

Por último, la búsqueda web nos muestra algo que no tiene mucho que ver con el diseño de las redes, sino con el crecimiento de ciertos servicios de Internet: hay mucho dinero en la publicidad. Éste es el motor económico que ha impulsado el crecimiento de la búsqueda web. El principal cambio de la publicidad impresa es la habilidad de dirigir los anuncios, dependiendo de lo que las personas estén buscando, para incrementar la relevancia de los mismos. Se utilizan variaciones de un mecanismo de subastas para asociar la consulta de búsqueda con el anuncio más valioso (Edelman y colaboradores, 2007). Desde luego que este nuevo modelo ha dado pie a nuevos problemas, como el **fraude del clic**, en el que los programas imitan a los usuarios y hacen clics en anuncios para provocar pagos que no se han obtenido en forma justa.

7.4 AUDIO Y VIDEO DE FLUJO CONTINUO

Las aplicaciones web y la web móvil no son los únicos desarrollos emocionantes en el uso de las redes. Para muchas personas, el audio y el video son el Santo Grial de las redes. Cuando se menciona la palabra “multimedia”, tanto los expertos como los ejecutivos empiezan a emocionarse al mismo tiempo. Los